
Golden-Section Search

Derivative-free optimization by interval reduction

Jonayed Mohiuddin

Student ID: 2105060

October 2026

Four questions

- 1 Why does the constant $0.618\dots$ appear?
- 2 How can one function value be reused at every step?
- 3 What accuracy is guaranteed after a fixed budget?
- 4 When should we *not* trust the answer?

The working viewpoint

The method does not chase a derivative. It shrinks a bracket that is known to contain one optimum.

SECTION 1

Problem Setting

Golden-Section Search | Jonayed Mohiuddin

A derivative-free question

$$x^* = \arg \max_{x \in [x_L, x_U]} f(x)$$

Only request: values of $f(x)$.

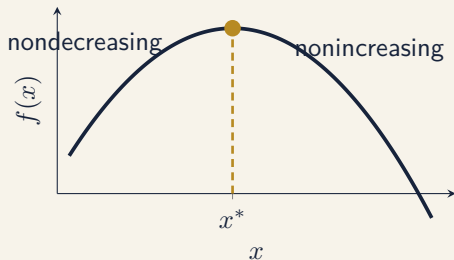
A reliable derivative may be unavailable, expensive, or meaningless.

Typical situations

- ▶ laboratory measurements;
- ▶ simulation runs;
- ▶ noisy or tabulated output.

For minimization, replace $\arg \max$ by $\arg \min$; the geometry is unchanged.

The unimodality contract



A maximum at x^* divides the bracket:

- ▶ f is nondecreasing up to x^* ;
- ▶ f is nonincreasing after x^* .

What is not required

No derivative, quadratic model, or smooth graph is required.

Source: Course notes, p. 1; maximization convention retained.

The bracket is a claim

- 1 **Valid domain:** every trial point is safe to evaluate.
- 2 **Contains the target:** the desired optimum is inside.
- 3 **One relevant mode:** one peak or trough governs the interval.

Why this matters

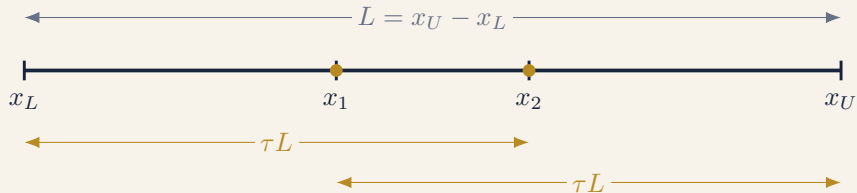
The method certifies a smaller *interval*; it cannot repair a bracket that excluded the optimum or mixed several extrema.

SECTION 2

The Search Mechanism

Golden-Section Search | Jonayed Mohiuddin

Two probes, one bracket



$$x_1 = x_U - \tau L$$

$$x_2 = x_L + \tau L$$

$$\tau = \frac{\sqrt{5} - 1}{2} \approx 0.618034, \quad x_L < x_1 < x_2 < x_U.$$

Source: Course notes, pp. 1-2. The derivation of τ follows in the analysis section.

Which side survives?

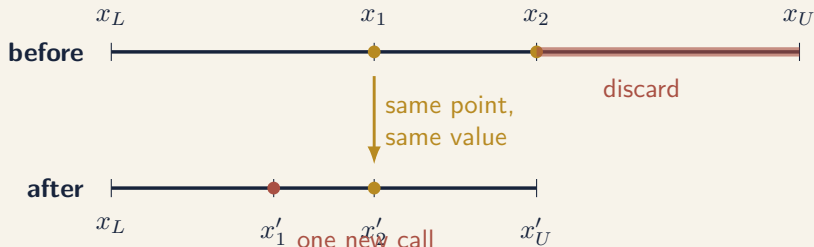
Comparison	Maximization keeps	Minimization keeps
$f(x_1) > f(x_2)$	$[x_L, x_2]$	$[x_1, x_U]$
$f(x_1) < f(x_2)$	$[x_1, x_U]$	$[x_L, x_2]$
$f(x_1) = f(x_2)$	$[x_1, x_2]$	$[x_1, x_2]$

Read one row, not the whole table

For a maximum, the larger value points toward the survivor. For a minimum, the smaller value does.

Source: The strict-inequality cases follow the update rule in the course notes, pp. 2-3.

Reuse one evaluation



For the illustrated maximization case, $f(x_1) > f(x_2)$. Therefore

$$x'_U \leftarrow x_2, \quad x'_2 \leftarrow x_1, \quad f(x'_2) \leftarrow f(x_1).$$

Only x'_1 is new, so only **one new function call** is needed.

Source: Course notes, p. 2.

Algorithm: initialize once

- 1 Set $\tau = (\sqrt{5} - 1)/2$ and $L = x_U - x_L$.
- 2 Form $x_1 = x_U - \tau L$ and $x_2 = x_L + \tau L$.
- 3 Evaluate $f_1 = f(x_1)$ and $f_2 = f(x_2)$.

State carried into the loop

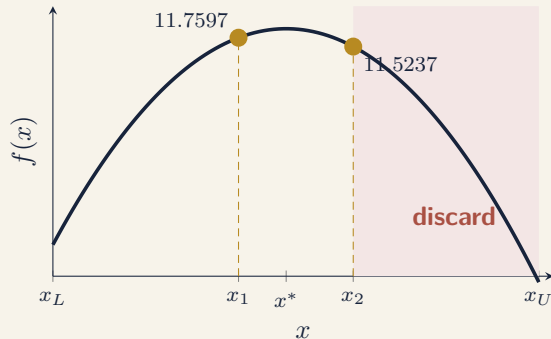
Keep the ordered endpoints, two probes, and their two function values. Losing a saved value also loses the main efficiency advantage.

Algorithm: repeat

- 1 Compare f_1 and f_2 .
- 2 Discard the side ruled out by unimodality.
- 3 Rename the surviving probe and reuse its value.
- 4 Create one new probe and evaluate f there.
- 5 Stop when the bracket meets the accuracy rule.

For minimization, reverse only the two strict comparison decisions; the probe geometry and reuse are unchanged.

Worked trace: first cut



Maximize

$$f(x) = 12 - (x - 2.4)^2$$

on $[0, 5]$.

$$x_1 = 1.90983, \quad x_2 = 3.09017.$$

Since

$$f(x_1) > f(x_2),$$

retain **$[0, 3.09017]$** .

Worked trace: cuts 0-2

k	Bracket before	x_1	x_2	Retained bracket
0	$[0, 5]$	1.90983	3.09017	$[0, 3.09017]$
1	$[0, 3.09017]$	1.18034	1.90983	$[1.18034, 3.09017]$
2	$[1.18034, 3.09017]$	1.90983	2.36068	$[1.90983, 3.09017]$

Watch the gold entry

It moves to the other probe column because the physical point and its function value are reused.

Worked trace: cuts 3-5

k	Bracket before	x_1	x_2	Retained bracket
3	[1.90983, 3.09017]	2.36068	2.63932	[1.90983, 2.63932]
4	[1.90983, 2.63932]	2.18847	2.36068	[2.18847, 2.63932]
5	[2.18847, 2.63932]	2.36068	2.46711	[2.18847, 2.46711]

Result after six comparisons

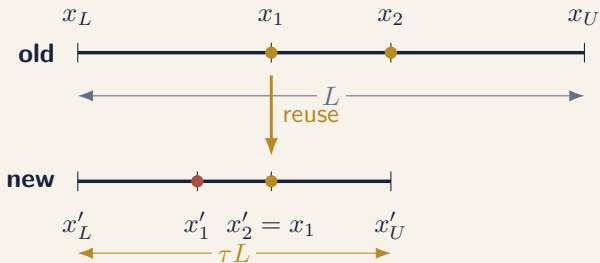
The true maximizer $x^* = 2.4$ remains inside $[2.18847, 2.46711]$, now only 0.27864 wide.

SECTION 3

Convergence and Cost

Golden-Section Search | Jonayed Mohiuddin

Why does 0.618 appear?



The old x_1 is $(1 - \tau)L$ from the left endpoint.

In the new bracket, the right probe is $\tau(\tau L) = \tau^2 L$ from that endpoint.

Reuse requires

$$1 - \tau = \tau^2.$$

Hence

$$\tau = \frac{\sqrt{5} - 1}{2}.$$

Source: Course notes, pp. 4-5; Kiefer (1953) formalized finite-budget sequential minimax search.

Why the discard is safe

Consider maximization with $x_1 < x_2$ and $f(x_1) > f(x_2)$.

- 1 Suppose, for contradiction, that $x^* > x_2$.
- 2 Both probes would then lie on the nondecreasing side of the peak.
- 3 Unimodality would require $f(x_1) \leq f(x_2)$, a contradiction.

Therefore $x^* \leq x_2$, so discarding $(x_2, x_U]$ preserves the invariant

$$x^* \in [x_L, x_U].$$

The other comparison and minimization cases are symmetric.

Rate and error guarantee

Exact contraction

$$L_k = \tau^k L_0.$$

For midpoint output,

$$|\hat{x}_k - x^*| \leq \frac{L_k}{2}.$$

Required reductions

$$k \geq \max \left\{ 0, \left\lceil \frac{\log(2\varepsilon/L_0)}{\log \tau} \right\rceil \right\}.$$

Example

If $L_0 = 5$ and $\varepsilon = 10^{-3}$, then $k = 17$. The actual bound is about 7.00×10^{-4} .

What actually converges?

Bracket width: exactly Q-linear,

$$\frac{L_{k+1}}{L_k} = \tau.$$

Location estimate: its guaranteed error is at most $L_k/2$, also decreasing linearly.

Objective value: unimodality alone gives no numerical rate. Extra regularity is needed. For example, if f is M -Lipschitz,

$$0 \leq f(x^*) - f(\hat{x}_k) \leq \frac{ML_k}{2}$$

for maximization. A smooth stationary optimum can yield an $O(L_k^2)$ objective error instead.

Count calls, not loops

Initialization

2 function calls

produce the first comparison.

With a stop-after-update implementation,

Each later comparison

1 new function call

because one value is reused.

$$n \text{ comparisons} \implies n + 1 \text{ evaluations.}$$

For an evaluation budget $E \geq 2$, the worst-case bracket is

$$L_{E-1} = \tau^{E-1} L_0.$$

This is why evaluation count is the meaningful complexity measure when f is a simulation or experiment.

Fixed-rule alternatives

- 1 **Equal-thirds search:** two new calls per comparison; retain $2/3$ of the bracket.
- 2 **Fibonacci search:** finite-budget minimax optimal; changing ratios require choosing the budget first.

Golden section's compromise: one stationary ratio, one reused value, and no predetermined final budget.

Source: Kiefer, "Sequential minimax search for a maximum," Proc. AMS 4 (1953), 502-506.

Brent's hybrid

- ▶ Propose an inverse parabolic interpolation step from sampled values.
- ▶ Accept it when safe; otherwise take a golden-section step.

Trade-off: golden section gives transparent worst-case contraction. Brent adds bookkeeping to gain speed when the curve is informative.

Source: SciPy 1.18 `minimize_scalar` documentation; Brent (1973).

SECTION 4

Reliable Computation

Golden-Section Search | Jonayed Mohiuddin

Stop safely near zero

The course-note relative test contains $x_U + x_L$ in the denominator. On a symmetric bracket such as $[-1, 1]$, it divides by zero.

For midpoint output, use the scale-aware condition

$$L_k \leq 2(\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max\{|x_L|, |x_U|\}).$$

It directly guarantees

$$|\hat{x}_k - x^*| \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max\{|x_L|, |x_U|\}.$$

Also stop on an evaluation budget or when rounding creates no distinct probe.

Source: Course notes, p. 3; compare SciPy's `absolute xatol` and `maxfun` controls.

Implementation guardrails

Preserve state

- ▶ move each (x_i, f_i) pair together;
- ▶ cache the surviving evaluation;
- ▶ assert $x_L < x_1 < x_2 < x_U$.

Exit honestly

- ▶ reject NaN or Inf values;
- ▶ stop if a probe rounds to an endpoint;
- ▶ return bracket, status, and call count.

Once machine arithmetic cannot create a distinct probe, further interval algebra cannot create new information.

How the method misleads

Several extrema

A legal-looking comparison can discard the globally best basin.

Noisy evaluations

One flipped comparison causes an irreversible wrong cut.

Flat optimum

The bracket shrinks, but the optimizer need not be unique.

Invalid values

NaN, Inf, or domain errors destroy the ordering decision.

Wrong inequality

A minimization update used for maximization walks away from the peak.

False precision

A tiny bracket does not correct model error or measurement bias.

Modern adaptations

Line search in many dimensions

$$\phi(\alpha) = F(\mathbf{x} + \alpha \mathbf{d})$$

turns one step-length choice into a scalar golden-section problem.

Noisy or expensive objectives

- ▶ cache results, repeat uncertain comparisons, and enforce a budget;
- ▶ stop when noise exceeds the improvement being resolved.

The method is sequential: one new call per step saves evaluations, but the next location must wait. Parallel hardware may favor a batched alternative.

SECTION 5

Mathematical Problems

Golden-Section Search | Jonayed Mohiuddin

SECTION 6

Application Problems

Golden-Section Search | Jonayed Mohiuddin

SECTION 7

Summary & References

Golden-Section Search | Jonayed Mohiuddin